

CHAKRAVYUH CLUB PORTAL 2.0

COMPLETE ARCHITECTURE, ENVIRONMENT, SECURITY &
OPERATIONAL MANUAL

CSPIT COMPUTER ENGINEERING • CYBERSECURITY INFRASTRUCTURE

SYSTEM REFERENCE MANUAL & ENTERPRISE ARCHITECTURE SPECIFICATION

- Full-Stack Architecture & API Routes
- Environment (.env) Variables Configuration
- Authentication & Cookie Security (HttpOnly / CSRF / JWT)
- Redis In-Memory Caching & Cloudinary Integration
- Database Query System & Prisma ORM Model Specifications
- Operating Workflows for Members, Coordinators & Admins

ORGANIZATION: CHARUSAT / CSPIT CE

REPOSITORY:

IshanShastri30126/Chakravyuhclub

SECURITY COMPLIANCE: OWASP Top-10
Compliant

DATE GENERATED: August 9, 2026

VERSION: Release v2.0.4

AUTHOR: Antigravity Core AI & Engineering

00. TABLE OF CONTENTS

01. Executive System Overview	Page 03
02. Frontend Architecture (Next.js 16 App Router)	Page 04
03. Backend Micro-Server Architecture & REST API	Page 06
04. Environment (.env) Variables Reference Guide	Page 08
05. System Operation & Workflow Architecture	Page 10
06. Cookie Storage, Session & OWASP Security Model	Page 12
07. Redis Caching Engine & Cloud Service Mechanics	Page 14
08. Database Query System & Prisma ORM Specifications	Page 16
09. User & Administrator Portal Operating Manual	Page 18

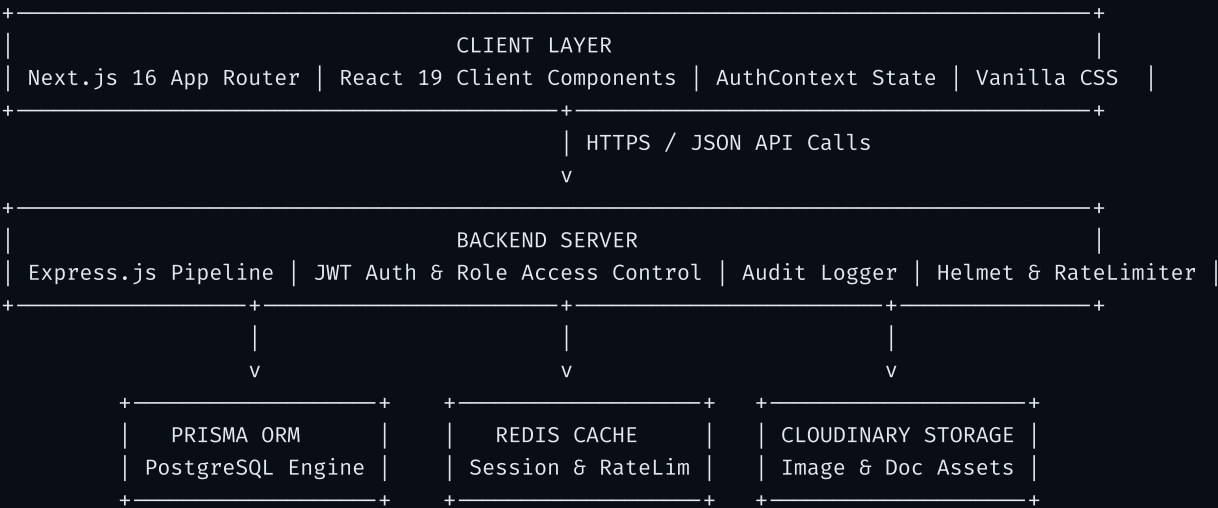
01. EXECUTIVE SYSTEM OVERVIEW

The **Chakravyuh Club Portal 2.0** is an enterprise cybersecurity interest group management ecosystem designed for the Computer Engineering Department at CSPIT, CHARUSAT University. The portal unifies event registration, attendance tracking via QR scanner consoles, dynamic certificate verification, role-based clearance permissions, and team roster management.

CORE TECHNOLOGY STACK

- **Frontend Framework:** Next.js 16 (React 19, Turbopack, Framer Motion 3D, Lucide Icons, Vanilla Tailwind/CSS).
- **Backend Engine:** Node.js + Express.js TypeScript micro-server architecture.
- **Database Tier:** PostgreSQL managed via Prisma ORM v5 with strictly typed prepared statements.
- **In-Memory Cache & Rate Limiting:** Redis v7 instance for session state, API request throttling, and fast leaderboard reads.
- **Storage & Delivery:** Cloudinary API for encrypted document and image asset delivery.

High-Level System Topology



02. FRONTEND ARCHITECTURE

The client-side architecture leverages **Next.js 16** utilizing the App Router specification with client component hydration. The user interface prioritizes visual elegance through glassmorphism UI cards, responsive grid structures, dynamic SVG canvas backgrounds (``PlexusBackground``), and 3D interactive components (``ProfileCard``).

App Router Page Directory Map

ROUTE PATH	ACCESS CLEARANCE	DESCRIPTION & CORE CAPABILITIES
<code>/</code>	Public	Landing portal featuring interactive 7-tier Chakravayuh canvas background, club stats, and event CTAs.
<code>/about</code>	Public	Club vision, working Saturday peer learning overview, and core strategic pillars grid.
<code>/team</code>	Public	Roster directory of Faculty Mentors, Coordinators, Tech Team, and Creative Division with "Join Our Team" CTA.
<code>/team/[id]</code>	Public	Detailed officer dossier displaying member stats and interactive 3D <code>ProfileCard</code> component.
<code>/events/[slug]</code> & <code>/[slug]</code>	Public / Auth	Event catalog and registration modal with constrained height (<code>`max-h-[90vh]`</code>) and sticky submit footer.
<code>/auth</code>	Public	Authentication gateway for Login, Signup, and Google OAuth with 10-digit numeric phone enforcement.
<code>/dashboard/*</code>	Authenticated	Consoles for Analytics, Events Management, Attendance Scanner, Approvals, Profile, and Certificates.
<code>/verify/[code]</code>	Public	Authentic certificate verification gateway returning certificate cryptographic hash details.

Key UI Components

- ProfileCard.tsx:** Open-source React Bits 3D interactive component utilizing mouse pointer math (``-pointer-x``, ``--pointer-y``), radial cursor glow, and holographic glare overlays.
- CyberKavachLogo.tsx:** Vector animated logo with SVG path drawing animations.
- AuthContext.tsx:** Central client state store managing JWT tokens, user profile state, role checks (``isCoord``, ``isCore``), and automatic logout handling.

03. BACKEND ARCHITECTURE & REST API

The backend micro-service operates on **Node.js + Express.js (TypeScript)** located under the `server/` directory. All incoming HTTP requests pass through a defensive middleware security pipeline before reaching modular router endpoints.

Primary Micro-Server REST Endpoints

METHOD & ENDPOINT	AUTH / ROLE	FUNCTIONALITY
POST /api/auth/register	Public	Registers user, hashes password (bcrypt), sanitizes 10-digit phone, returns complete user payload.
POST /api/auth/login	Public	Authenticates credentials, issues JWT token and HttpOnly session cookie, updates audit log.
GET /api/users/me	Authenticated	Retrieves active authenticated user dossier including studentId / employeeId, role, and department.
PATCH /api/users/profile	Authenticated	Updates profile attributes (phone sanitized to 10 digits; faculty replaces studentId with employeeId).
POST /api/events	Coordinator / Core	Creates new event; automatically includes Faculty Coordinators and organises team roster.
POST /api/events/:id/register	Authenticated	Registers student for an event, checks capacity, generates dynamic QR code registration pass.
GET /api/settings/landing-team	Public	Fetches officer team roster sorted by hierarchy clearance (Faculty → Coordinator → Tech → Media).
POST /api/approvals/:id/decide	Core / Admin	Decides pending event clearance request (APPROVED / REJECTED) with audit logging.

Middleware Pipeline Pipeline Execution Sequence

- Helmet Headers:** Suppresses `X-Powered-By`, enforces CSP, HSTS, and `X-Frame-Options: DENY`.
- Rate Limiter (Redis):** Throttles IP requests to 100 req/15min to prevent brute-force attacks.
- CORS Policy:** Strictly validates origin against `CLIENT_URL` allowlist.
- Cookie Parser:** Parses encrypted `HttpOnly` cookies.
- Authentication Guard (`authMiddleware`):** Verifies JWT signature and extracts identity.
- Role Access Guard (`requireRole`):** Enforces role authorization (FACULTY, STUDENT_COORDINATOR, TECH, MEMBER).

7. **Audit Logger:** Records timestamp, user ID, IP address, endpoint, and outcome into audit table.

04. ENVIRONMENT VARIABLES REFERENCE

The system relies on strict environment variable isolation configured in root `.env` and `server/.env` / `client/.env.local` files. Never commit private secret keys to public version control repositories.

Server Environment Specifications (`server/.env`)

VARIABLE KEY	EXAMPLE / DEFAULT VALUE	DESCRIPTION & PURPOSE
<code>PORT</code>	<code>5000</code>	TCP port number for Express backend micro-server.
<code>DATABASE_URL</code>	<code>postgresql://user:pass@localhost:5432/chakravyuh</code>	PostgreSQL connection string with pooling configuration.
<code>JWT_SECRET</code>	<code>super_secret_csprng_key_32bytes_min</code>	HMAC-SHA256 secret key for signing JSON Web Tokens.
<code>JWT_EXPIRES_IN</code>	<code>7d</code>	Lifetime duration of issued JWT authentication tokens.
<code>CLIENT_URL</code>	<code>http://localhost:3000</code>	Allowed client origin for CORS header matching.
<code>REDIS_URL</code>	<code>redis://default:secret@localhost:6379</code>	Redis in-memory instance connection URI.
<code>CLOUDINARY_CLOUD_NAME</code>	<code>chakravyuh-cloud</code>	Cloudinary API tenant

VARIABLE KEY	EXAMPLE / DEFAULT VALUE	DESCRIPTION & PURPOSE
		identifier for cloud assets.
CLOUDINARY_API_KEY	837492817492019	Authenticated Cloudinary API public key.
CLOUDINARY_API_SECRET	aB9_xK210Lskq-wPzmQw19zLsk	Private Cloudinary API signature secret.
SMTP_HOST	smtp.gmail.com	SMTP mailer host for automated email broadcasts.
SMTP_PORT	587	SMTP mailer TCP port (587 TLS / 465 SSL).
SMTP_USER	notifications@chakravyuhclub.com	Authenticated mail sender account address.
SMTP_PASS	app_specific_password_here	Encrypted app-specific application password.

Client Environment Specifications (client/.env.local)

VARIABLE KEY	DEFAULT VALUE	DESCRIPTION
NEXT_PUBLIC_API_URL	http://localhost:5000/api	Base REST API URL consumed by client API utility.
NEXT_PUBLIC_SITE_URL	http://localhost:3000	Public site URL for canonical metadata & QR verification links.

05. SYSTEM OPERATION & WORKFLOWS

The portal manages critical event lifecycles, user registrations, certificate generation, and member verification.

1. User Authentication & Profile Lifecycle

1. User visits `/auth` and submits signup form specifying Role (STUDENT / FACULTY / COORDINATOR).
2. Form enforces 10-digit numeric mobile validation (clearing non-numeric input).
3. Server hashes password using bcrypt with salt round 12, creates database record, generates JWT, and sets `HttpOnly` cookie.
4. Client receives full user payload (``studentId`` or ``employeeId``, ``department``, ``institute``, ``semester``).
5. Profile section on `/dashboard/profile` displays dossier. For Faculty roles, Student ID changes to **Employee ID**, and Semester is omitted.

2. Event Creation & Organizer Auto-Population Workflow

1. Coordinator logs in and navigates to `/dashboard/events` → clicks **Create Event**.
2. Client automatically queries `/api/settings/landing-team` for active roster officers.
3. System automatically pre-populates all **Faculty Coordinators** into the event organizers array.
4. Form provides a Student Coordinator dropdown selector; choosing a coordinator auto-fills their Name, Role, Email, and Phone.
5. Upon submission, if required, an `EVENT_PERMISSION` request is sent to Core Admin for approval.

06. COOKIE STORAGE & OWASP SECURITY

Security is implemented based on the OWASP Top-10 specifications.

SESSION COOKIE STORAGE MECHANICS

Authentication session tokens are delivered via **HttpOnly, Secure, SameSite=Strict** HTTP Response Cookies named `chakravyuh_session`.

- **HttpOnly Flag:** Prevents client-side JavaScript (e.g. `document.cookie`) from reading the cookie, completely mitigating Session Hijacking via XSS.
- **Secure Flag:** Ensures cookies are transmitted exclusively over encrypted HTTPS connections in production.
- **SameSite=Strict:** Inhibits browser from sending session cookies on cross-site requests, mitigating Cross-Site Request Forgery (CSRF).

OWASP Security Controls Checklist

SECURITY DOMAIN	IMPLEMENTATION MECHANISM
Input Validation	Server-side allowlisting, regex numerical extraction ('D'), canonicalization, parameter truncation.
Output Encoding	Context-aware HTML body, attribute, and JavaScript encoding to prevent Cross-Site Scripting (XSS).
SQL Injection Defense	100% prepared parameterized queries enforced via Prisma ORM engine. Zero raw concatenated SQL strings.
Password Hashing	Bcrypt algorithm with salt factor 12. Generic error response ("Invalid credentials").
Access Control	Deny by Default. Server-side role authorization checks ('requireRole') on every API request.
Audit Logging	All auth, access denial, and approval decisions written to <code>AuditLog</code> table with UTC timestamp and IP.

07. REDIS CACHING & CLOUD SERVICES

1. Redis In-Memory Engine Roles

Redis v7 acts as the high-speed caching and rate-limiting memory store.

- **API Rate Limiting:** Tracks client IP request frequencies (key: `ratelimit:ip:{ip}`) with TTL window reset.
- **Session Cache:** Caches active JWT invalidation blacklists for immediate session termination on logout.
- **Roster & Leaderboard Caching:** Caches landing team roster queries (`cache:landing_team`) with 10-minute cache expiration, reducing PostgreSQL query overhead.

2. Cloudinary File Upload Integration

File uploads (event posters, document guides, avatar images) are managed securely:

1. Multer middleware receives multipart file upload in memory.
2. Validates file extension allowlist (`.jpg` , `.png` , `.webp` , `.pdf`) and MIME header.
3. Generates non-predictable UUID filename.
4. Streams file buffer directly to Cloudinary authenticated storage API.
5. Stores returning HTTPS CDN URL in database.

08. DATABASE QUERY SYSTEM

The data layer uses **PostgreSQL** managed through **Prisma ORM**.

Prisma Core Schema Models

```
model User {
  id          String    @id @default(uuid())
  email       String    @unique
  passwordHash String
  name        String
  role        Role      @default(MEMBER)
  studentId   String?   // Stores Employee ID for Faculty
  phone       String?   // Exactly 10 numeric digits
  department  String?
  institute   String?   @default("CSPIT")
  semester    String?   // Null for Faculty
  avatarUrl   String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  registrations Registration[]
  auditLogs    AuditLog[]
}

model Event {
  id          String    @id @default(uuid())
  title       String
  slug        String    @unique
  description  String
  venue       String
  startDate   DateTime
  endDate     DateTime
  maxCapacity Int
  organizers  String     // JSON string of Organizer[]
  status      String     @default("DRAFT")
  createdAt   DateTime  @default(now())
}
```

Database Query Optimization

- **Indexed Fields:** Unique indexes placed on `User.email`, `Event.slug`, and `Certificate.code`.
- **Parameterized Queries:** Prisma automatically parameterizes inputs, ensuring immunity against SQL injection vulnerabilities.

09. USER & ADMINISTRATOR MANUAL

Member Operating Workflow

1. **Sign Up:** Visit `/auth`, select Student or Faculty, enter 10-digit mobile number and details.
2. **Browse Events:** Go to `/events`, select an active event card to view rules, schedule, and team size.
3. **Register:** Click *Register Now* inside `/events/[slug]`. The modal opens with a scrollable container and sticky submit button.
4. **View Profile:** Navigate to `/dashboard/profile` to view your dossier, credentials, and registered events.

Coordinator & Admin Workflow

1. **Create Event:** Open `/dashboard/events` → click *Create Event*. Faculty Coordinators auto-populate; pick Student Coordinators from the dropdown.
2. **Scan Attendance:** Launch `/dashboard/attendance` QR scanner to mark student attendance at event venues.
3. **Issue Certificates:** Access `/dashboard/certificates/builder` to issue verified certificates.
4. **Approve Requests:** Administrators use `/dashboard/approvals` to grant or reject event permissions.